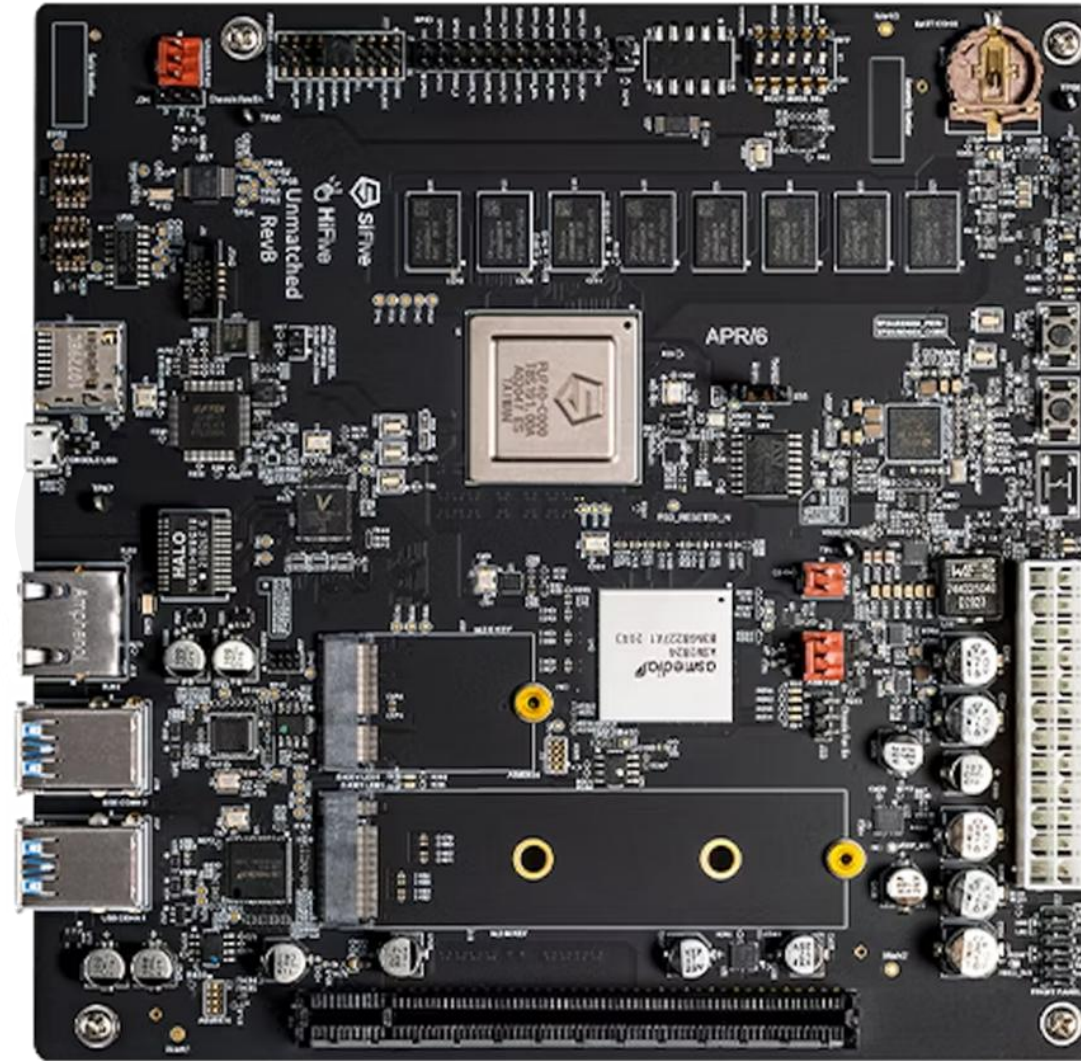


CS-301 Microprocessor Based System Design

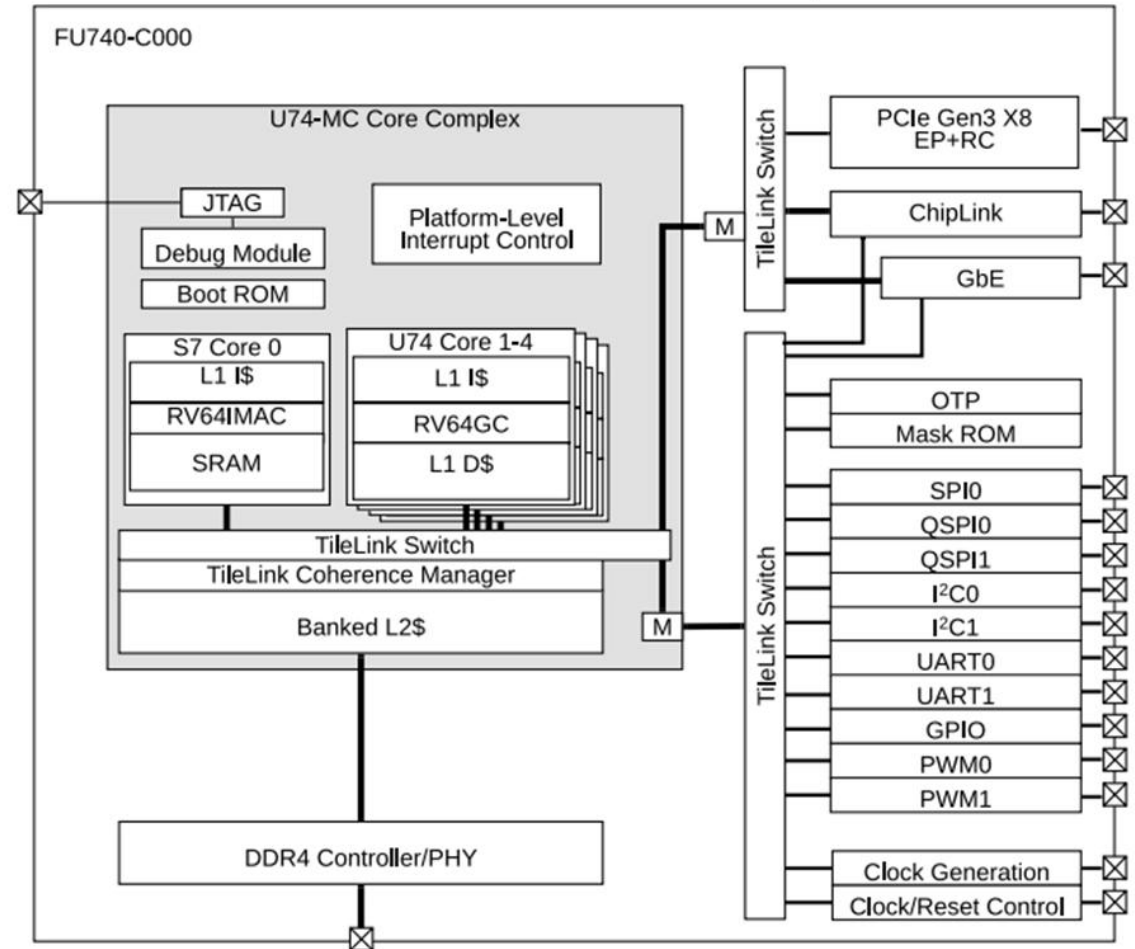
Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 8

HiFive Unmatched RevB



Freedom U740 SoC by SiFive

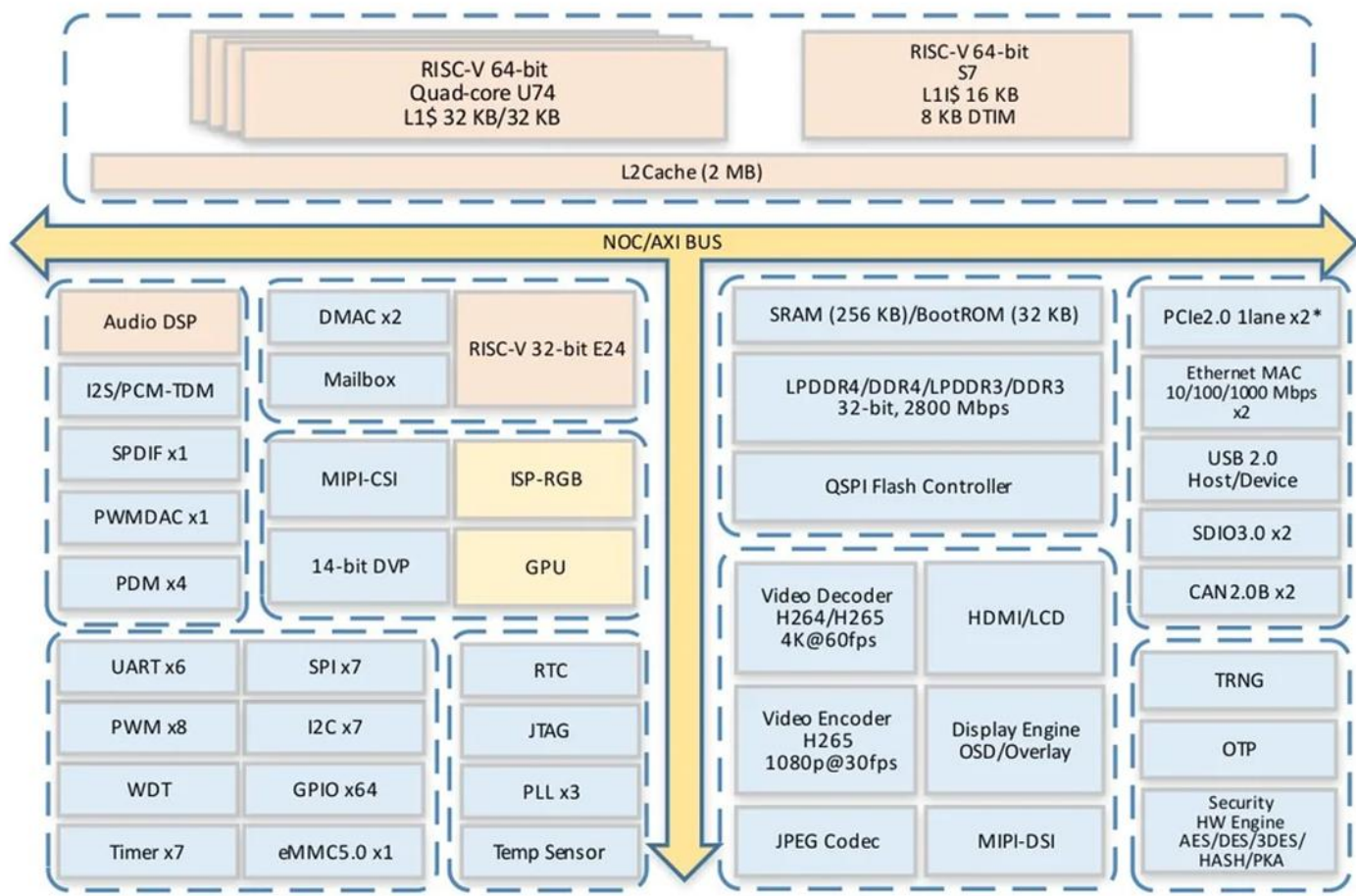


PineTab-V by PINE64

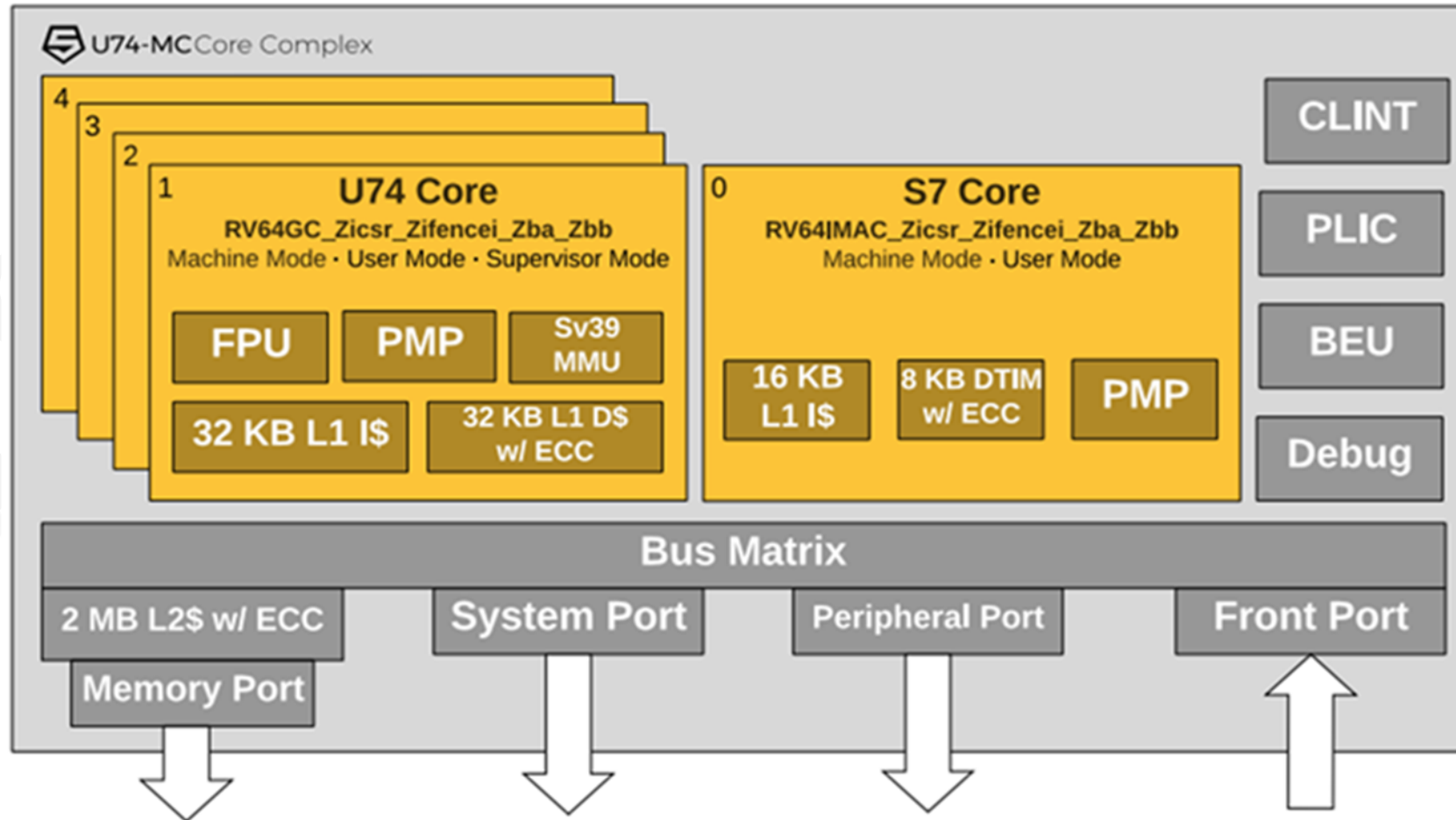
- One of the first RISC-V tablets



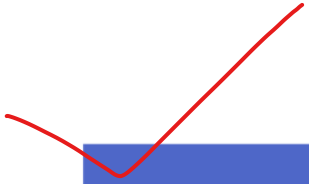
JH7110 SoC by StarFive



SiFive U74-MC

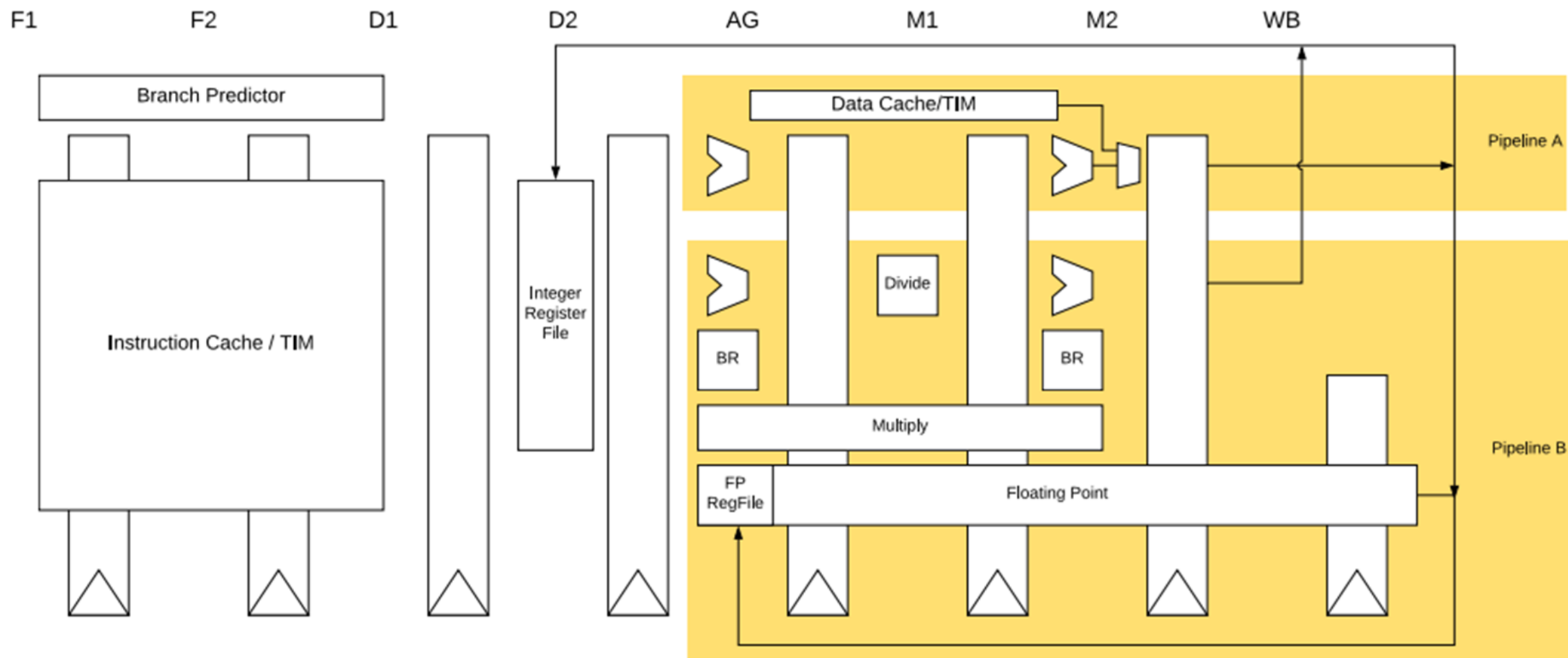


U74 RISC-V Core

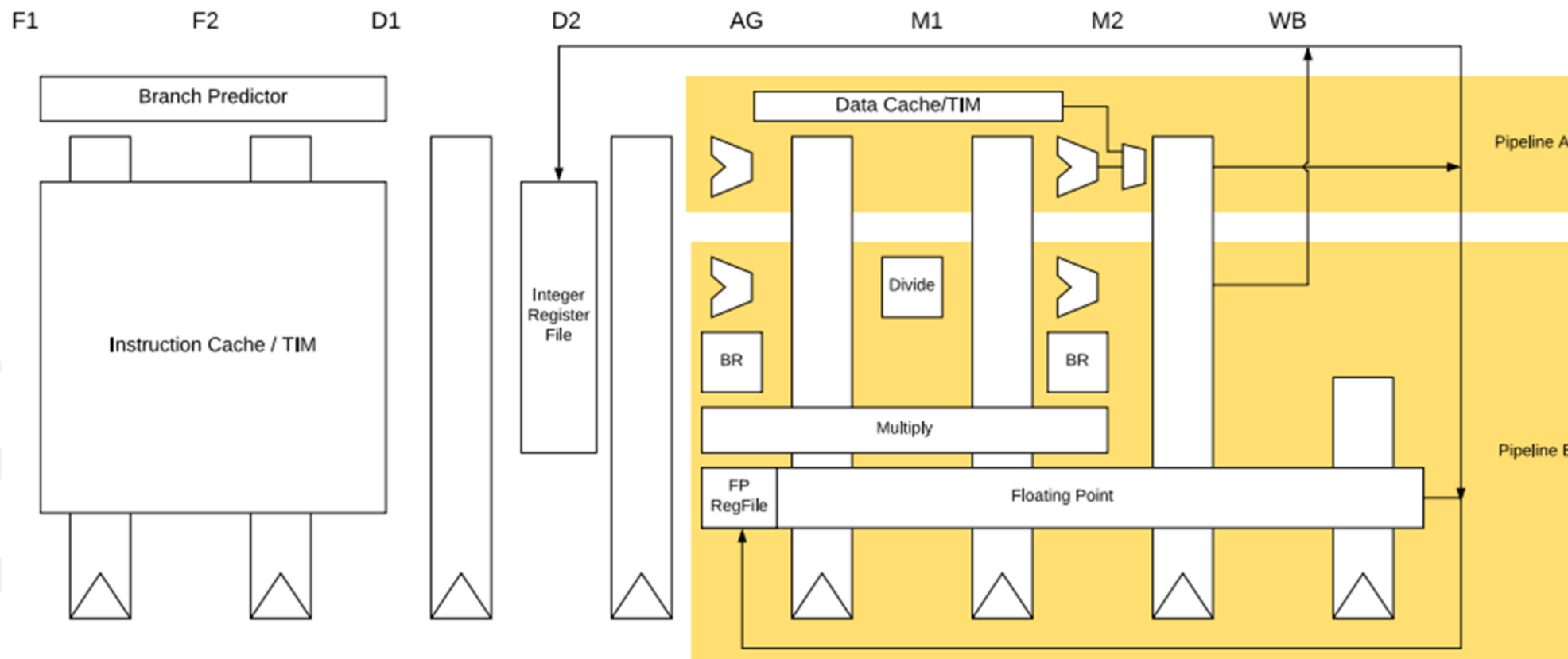


Feature	Description
ISA	RV64GC
Modes	Machine mode, User mode, Supervisor mode
L1 Instruction Cache	32 KiB 4-way instruction cache
L1 Data Cache	32 KiB 8-way data cache
L2 Cache	2 MiB 16-way L2 cache with 4 banks
ECC Support	Single error correction, double error detection on the data cache and L2 cache
Floating Point	IEEE 754 Floating Point Unit
Fast I/O	Present
Physical Memory Protection	8 regions with a granularity of 4096 bytes
Memory Management Unit	Sv39 virtual memory support with fully-associative 40-entry L1 Data and Instruction TLBs, and a direct-mapped 512-entry L2 TLB.

U74 RISC-V Processor Pipeline



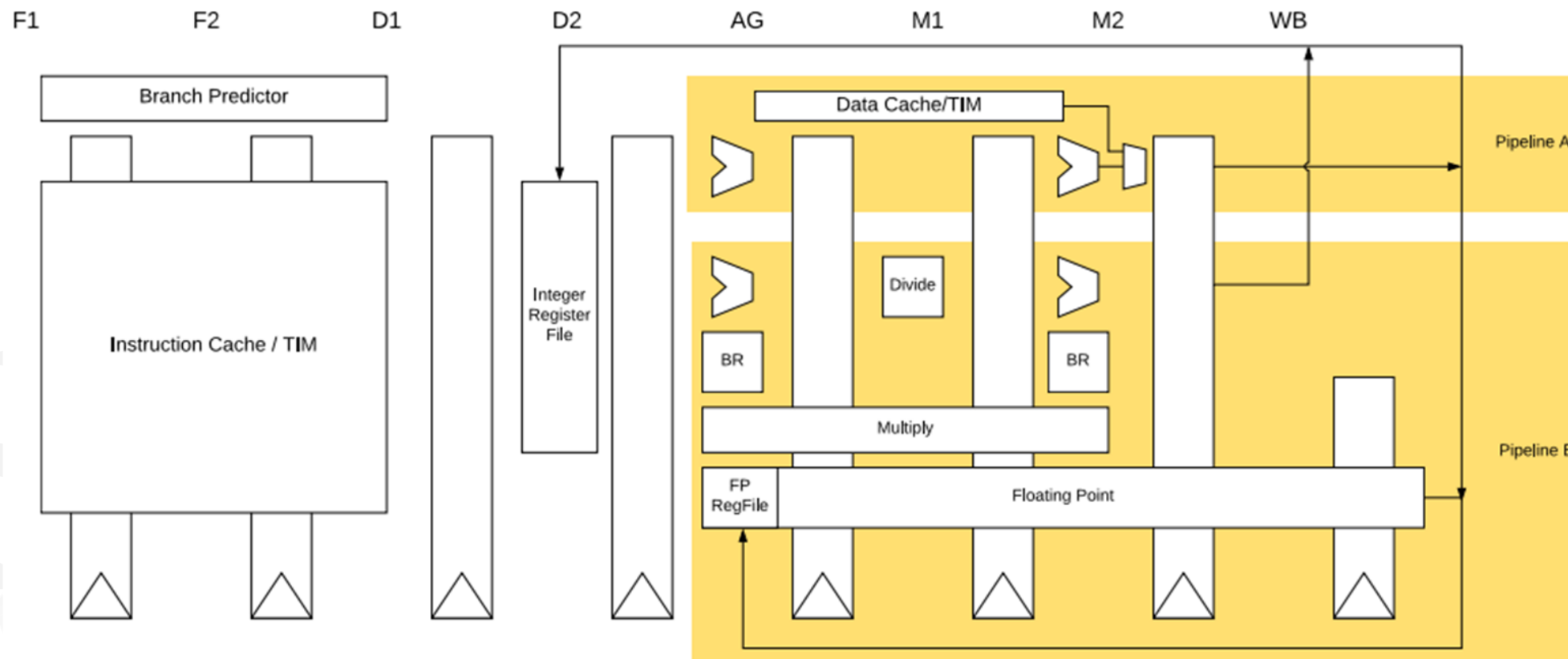
U74 RISC-V Processor Pipeline



The U74 execution unit is a dual-issue, in-order pipeline. The pipeline comprises eight stages:

- two stages of instruction fetch (F1 and F2),
- two stages of instruction decode (D1 and D2),
- address generation (AG),
- two stages of data memory access (M1 and M2),
- and register write-back (WB).

U74 RISC-V Processor Pipeline



- Pipeline A: Primarily handles loads, stores, and integer-to-floating-point moves. It can also handle standard integer ALU operations.
- Pipeline B: Used for branches, multiplications, divisions, and floating-point operations. Integer ALU instructions can be executed in either pipeline.

U74 RISC-V Processor Pipeline

- The pipeline has a peak execution rate of two instructions per clock cycle, and is fully bypassed (forwarding) so that most instructions have a one-cycle result latency: Bypassing (forwarding) = sending result directly to next pipeline stage without waiting for register writeback
 - Meaning: Integer arithmetic and branch instructions can execute in either the AG or M2 pipeline stage.
 - If such an instruction's operands are available when the instruction enters the AG stage, then it executes in AG; otherwise, it executes in M2.
 - Loads produce their result in the M2 stage.
 - There is no load-use delay for most integer instructions. However, effective addresses for memory accesses are always computed in the AG stage.
 - Hence, loads, stores, and indirect jumps require their address operands to be ready when the instruction enters AG. If an address-generation operation depends upon a load from memory, then the load-use delay is two cycles.

U74 RISC-V Processor Pipeline

- Integer multiplication instructions consume their operands in the AG stage and produce their results in the M2 stage. ag is where computation begins
 - The integer multiplier is fully pipelined. see multiplication unit start at ag and ends at M2
- Integer division instructions consume their operands in the AG stage.
 - These instructions have between a six-cycle and 68-cycle result latency, depending on the operand values.
 - The latency of DIV, DIVU, REM, and REMU instructions can be determined by calculating:

$$\text{Latency} = 2 \text{ cycles} + \log_2(\text{dividend}) - \log_2(\text{divisor}) + 1 \text{ cycle if the input is negative} + 1 \text{ cycle if the output is negative}$$
- CSR accesses execute in the M2 stage. csr is done using its relative instruction
 - CSR read data can be bypassed to most integer instructions with no delay.
 - Most CSR writes flush the pipeline, which is a seven-cycle penalty

since, result says new instruction to fetch so prefetched 7 instructions (since, 8-1) stages to be flushed

U74 RISC-V Processor Pipeline

- The pipeline implements a flexible dual-instruction-issue scheme. Provided there are **no data hazards** between a pair of instructions, the two instructions may issue in the same cycle, provided the following constraints are met:
data hazard, instr depends on the result of prev instr
 - At most one instruction accesses data memory.
 - At most one instruction is a branch or jump.
 - At most one instruction is a floating-point arithmetic operation.
 - At most one instruction is an integer multiplication or division operation.
 - Neither instruction explicitly accesses a CSR. CSR instructions are issued alone
- The pipeline only interlocks (automatically insert stalls) on read-after-write and write-after-write hazards, so instructions may be scheduled to avoid stalls i.e. compilers should reschedule the code to avoid stalls.
avoiding stalls by rescheduling the code

U74 Pipeline

- Instruction Fetch Unit

- The instruction fetch unit delivers up to 8 bytes of instructions per clock cycle to support superscalar instruction execution. 4 bytes each instruction. dual issue hence 8 bytes
- Fetches are always word-aligned and there is a one cycle penalty for branching to a 32-bit instruction that is not word-aligned. word aligned means, address always multiple of 4 since 4Bytes address.
- For compressed instructions four 16-bit instructions can be fetched per cycle, which means the instruction fetch unit can be idle when executing programs comprised mostly of compressed 16-bit instructions. This reduces memory accesses and power consumption
- All branches must be aligned to half-word addresses. Otherwise, the fetch generates an instruction **address misaligned trap**.
- Trying to fetch from a non-executable or unimplemented address results in an **instruction access trap**.

Superscalar Instruction Execution

A superscalar CPU can execute multiple instructions in the same clock cycle.

CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture - 9

U74 Instruction Fetch Unit

F1:

- send PC to I-cache
- query BTB, BHT, RAS, IJTP
- compute multiple speculative PCs

F2:

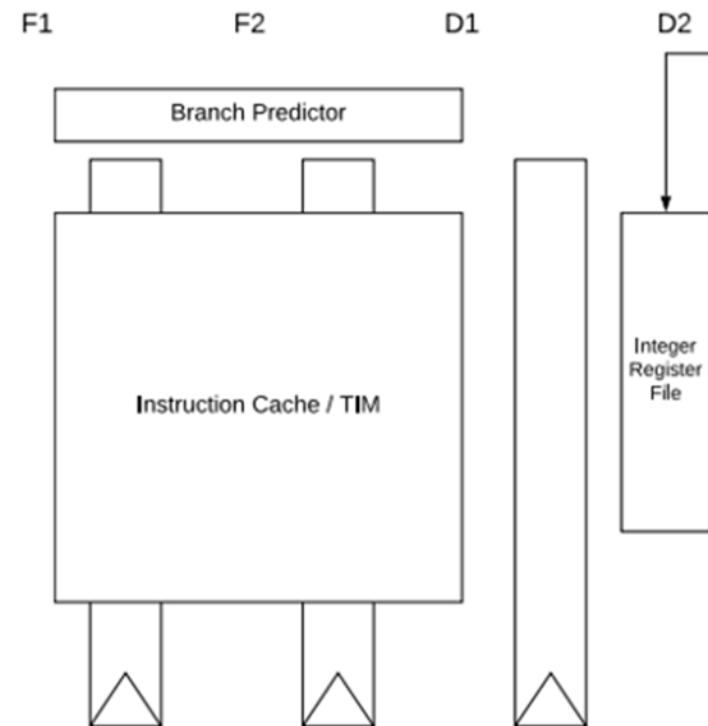
- receive instruction
- extract opcode, rd, rs1
- classify instruction (branch/jump/return/etc.)
- validate prediction
- finalize next PC

D1/D2:

- full decode

F1 outputs:

- predicted_taken (from BHT)
- predicted_target (from BTB / IJTP / RAS)
- sequential PC (PC + 4 or +2)



U74 Pipeline

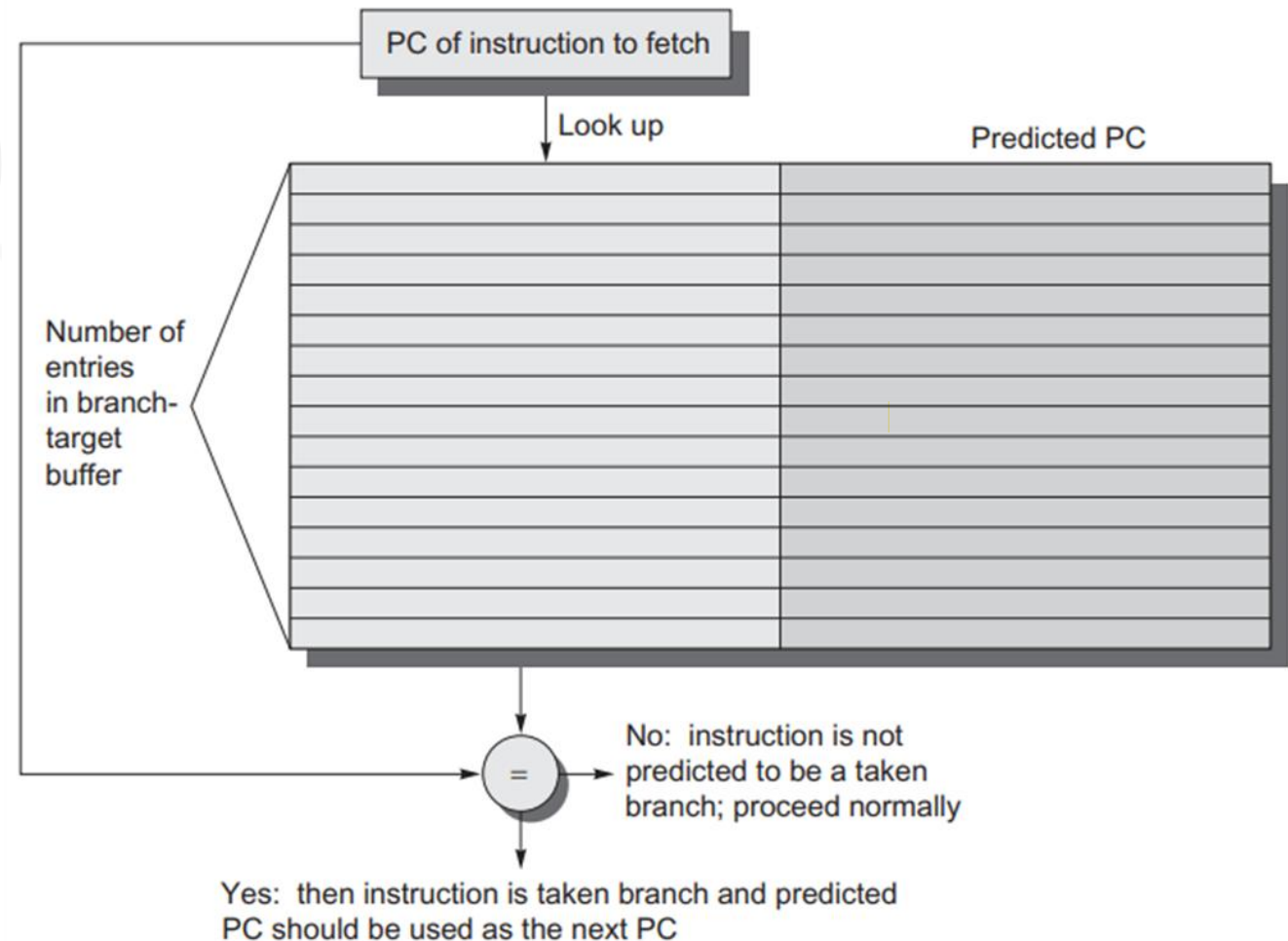
- Branch Prediction
 - The branch prediction hardware consists of
 - A 16-entry branch target buffer (BTB), which predicts the target of taken branches and direct jumps;
 - The BTB has one-cycle latency, so that correctly predicted branches and direct jumps result in no penalty, provided the target is 8-byte aligned
 - Direct jumps that miss in the BTB result in a one-cycle fetch bubble
 - A 3.6 KiB branch history table (BHT), which predicts the direction of conditional branches;
 - The BHT is a correlating predictor that supports long branch histories (e.g. gshare)
 - An 8-entry indirect-jump target predictor (IJTP);
 - A 6-entry return-address stack (RAS), which predicts the target of procedure returns.

U74 Pipeline

- Branch Prediction decoupled = not waiting for each other
 - The instruction fetch unit is decoupled from the execution unit, so that correctly predicted control-flow events usually do not result in execution stalls. because correct instr already fetched due to prediction
 - The IJTP, and RAS take precedence over the BTB. If these structures' predictions disagree with the BTB's prediction, a one-cycle fetch bubble results.
 - BTB is purely associative (PC-based guess) others are semantic predictors (based on instruction type) that's why BTB has less priority.
 - A fetch bubble but might not result in an execution stall if the fetch queue is sufficiently full
 - Mispredicted branches usually incur a four-cycle penalty, but sometimes the branch resolves later in the execution pipeline and incurs a six-cycle penalty instead.
 - Mispredicted indirect jumps incur a six-cycle penalty.
purely associative = search for matching pc tags

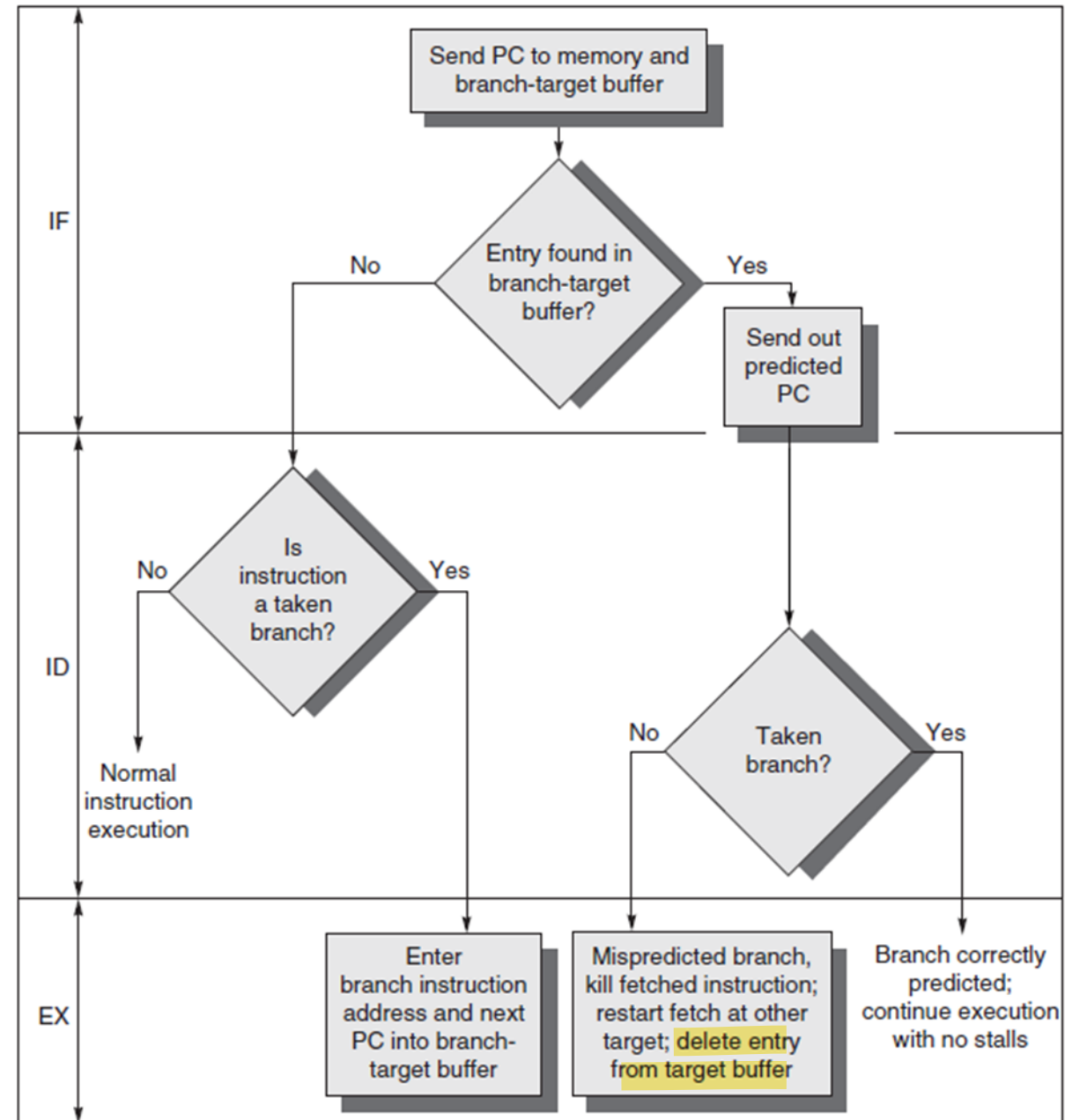
Branch Target Buffer

- If the instruction is a branch and we know what the next PC should be, we can have a branch penalty of zero.
- A memory that stores the predicted address for the next instruction after a taken branch is called a **branch-target buffer (BTB)**.
- If the PC of the fetched instruction matches an address in the prediction buffer, then the corresponding predicted PC is used as the next PC.
- The hardware for this branch-target buffer is essentially identical to the hardware for a cache. i.e. it could have different mappings and replacement policies.



Branch Target Buffer

- Branch-target buffer predicts the next instruction address and will send it out before decoding the instruction
- If the PC of the fetched instruction matches an address in the prediction buffer, then fetching begins immediately and the corresponding predicted PC is used as the next PC.
- We only need to store the predicted-taken branches in the branch target buffer, since an untaken branch should simply fetch the next sequential instruction, as if it were not a branch.



Correlating Predictors - gshare

- XOR Hashing: gshare works by XORing the Program Counter (PC) with the Global History Register (GHR).
- Aliasing Reduction: Standard correlating predictors can suffer from "aliasing," where different branches map to the same table entry.
- The XOR function in gshare spreads the index more evenly across the Branch History Table (BHT), making better use of the available hardware.

